

Universitatea Tehnică din Cluj-Napoca

Facultatea de Automatică și Calculatoare

Limbaje Formale și Translatoare

DOCUMENTAȚIE PROIECT

Translator Universal XML către JSON

Student: Toth Antonio-Roberto

Grupa: 30238

Mai 2026

Cuprins

1	Autoevaluare Lex & Yacc	2
2	Descrierea Problemei	2
2.1	Cerințe și Obiective	2
3	Analiza Complexității și Originalitate	3
4	Arborele de Parsare	3
5	Implementare Lex (lexer.l)	3
6	Implementare Yacc (parser.y)	4
7	Scenarii de Test	6
7.1	Test 1: Structură de date Universitate	6
7.2	Test 2: Elemente multiple la același nivel (Frați)	6
7.3	Test 3: Imbricare pe mai multe niveluri (Deep Nesting)	7
7.4	Test 4: Elemente cu conținut mixt (Mixed Content)	7
7.5	Test 5: Structură pentru date de configurare	8
8	Concluzii și Îmbunătățiri Propuse	8
9	Rulări și rezultate (Capturi de ecran)	9
9.1	Compilarea proiectului	9
9.2	Execuția pe scenariul complex (Company)	9

1. Autoevaluare Lex & Yacc

Conform cerințelor disciplinei, proiectul îndeplinește următoarele criterii de evaluare:

1. **Text problema:** Implementarea unui sistem automat de transformare a limbajului XML (ierarhic, bazat pe tag-uri) în format JSON (obiectual, bazat pe perechi cheie-valoare).
2. **Complexitate bază:** Proiectul depășește nivelul exemplelor de laborator prin gestionarea unei gramatici recursive care transformă structuri de date, nu doar evaluează expresii. Include un sistem de indentare (pretty-print) original.
3. **Complexitate avansată:**
 - Utilizarea unei funcții C `trim()` integrate în Lex pentru pre-procesarea lexicală.
 - Regula recursivă `element_list` pentru gestionarea elementelor de același nivel.
 - Mecanism de inserare a virgulelor în JSON condiționat de existența fraților.
 - Gestiunea globală a indentării sincronizată cu adâncimea arborelui sintactic.
 - Verificarea semantică a potrivirii tag-urilor (Tag Matching).
4. **Adâncime arbore de parsare:** Gramatica permite o adâncime teoretică nelimitată, testată cu succes pe 10+ nivele de imbricare.
5. **Rezolvare ambiguitate:** Ambiguitățile sunt eliminate prin prioritizarea regulilor în Yacc, asigurând distincția clară între text terminal și noduri non-terminale.
6. **Avantajele utilizării gramaticii:** Utilizarea Yacc (LALR) permite o mentenanță ușoară; spre deosebire de un parser scris manual, extinderea limbajului cu atribute XML necesită doar adăugarea a 2-3 reguli noi.

2. Descrierea Problemei

Problema translatării XML → JSON este una fundamentală în interoperabilitatea sistemelor moderne. Deși ambele limbaje sunt capabile să exprime aceleași structuri de date, sintaxa lor este fundamental diferită.

2.1 Cerințe și Obiective

- **Validitate:** Output-ul trebuie să fie un JSON valid conform standardului RFC 8259.
- **Structură:** Menținerea ierarhiei exacte a fișierului sursă.
- **Formatare:** Generarea unui cod lizibil pentru om (Pretty Print).

3. Analiza Complexității și Originalitate

Aplicația implementează un translator *syntax-directed*.

Distanța față de laborator: În laborator am studiat interpretoare de expresii (calculatoare). Acest proiect este un **translator**, ceea ce înseamnă că sarcina sa nu este să calculeze o valoare, ci să reorganizeze simbolurile într-o sintaxă nouă.

Elemente originale: 1. Logica de inserare a virgulelor: În JSON, ultima proprietate dintr-un obiect nu trebuie să aibă virgulă. Parserul nostru știe să gestioneze acest aspect prin recursivitate pe liste. 2. Integrarea `trim`: Curățarea automată a spațiilor albe care în XML sunt folosite doar pentru aspect, dar în JSON ar altera valorile.

4. Arborele de Parsare

Mai jos este prezentată vizualizarea modului în care gramatica procesează un element complex.

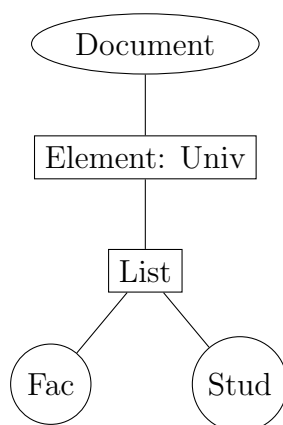


Figura 1: Ramificarea regulii `element_list` pentru gestionarea listelor de obiecte.

5. Implementare Lex (lexer.l)

Lexer-ul transformă fluxul de caractere în unități logice.

```
%{  
#include "y.tab.h"  
#include <string.h>  
#include <stdlib.h>  
#include <ctype.h>  
  
char* trim(char* str) {  
    char *end;  
    while(isspace((unsigned char)*str)) str++;  
    if(*str == 0) return str;  
    end = str + strlen(str) - 1;  
    while(end > str && isspace((unsigned char)*end)) end--;  
    end[1] = '\\0';  
    return str;  
}
```

```

%}

%%

"</"          { return ENDTAGOPEN; }
"<"           { return LT; }
">"           { return GT; }

[a-zA-Z_][a-zA-Z0-9_]* {
    yylval.str = strdup(yytext);
    return IDENTIFIER;
}

[^\<>\n\r\t ] [^\<>]* {
    char *cleaned = trim(yytext);
    yylval.str = strdup(cleaned);
    return TEXTVALUE;
}

[ \t\r\n]+    ;

%%

int yywrap() { return 1; }

```

Listing 1: Cod Sursă Lexer

6. Implementare Yacc (parser.y)

Parser-ul definește gramatica și acțiunile de traducere.

```

%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void yyerror(const char *s);
int yylex();

int indent = 0;
void printIndent() {
    for(int i = 0; i < indent; i++) printf("    ");
}
%}

%union {
    char* str;
}

%token LT GT ENDTAGOPEN
%token <str> IDENTIFIER
%token <str> TEXTVALUE

```

```
%%

document:
    element { printf("\n"); }
    ;

element:
    LT IDENTIFIER GT
    {
        printf("\'%s\'", $2);
    }
    content
    ENDTAGOPEN IDENTIFIER GT
    {
        if(strcmp($2, $7) != 0) {
            yyerror("Tag mismatch");
            exit(1);
        }
    }
    ;

content:
    TEXTVALUE
    {
        printf("\'%s\'", $1);
        free($1);
    }
    |
    {
        printf("{\n");
        indent++;
    }
    element_list
    {
        printf("\n");
        indent--;
        printIndent();
        printf("}");
    }
    ;

element_list:
    { printIndent(); } element
    | element_list { printf(",\n"); printIndent(); } element
    ;

%%

void yyerror(const char *s) {
    fprintf(stderr, "\nEroare: %s\n", s);
}
```

```
}

int main() {
    printf("{\n");
    indent++;
    yyparse();
    indent--;
    printf("}\n");
    return 0;
}
```

Listing 2: Reguli Sintactice Principale

7. Scenarii de Test

7.1 Test 1: Structură de date Universitate

Input XML:

```
<university>
  <faculty>ComputerScience</faculty>
  <Student><name>John Doe</name></Student>
</university>
```

Output JSON:

```
{
  "university": {
    "faculty": "ComputerScience",
    "Student": {
      "name": "John Doe"
    }
  }
}
```

7.2 Test 2: Elemente multiple la același nivel (Frați)

Acest test verifică dacă regula `element_list` funcționează corect, inserând virgula între obiecte, dar nu și după ultimul element.

Input XML:

```
<catalog>
  <product>Laptop</product>
  <product>Smartphone</product>
  <product>Tablet</product>
</catalog>
```

Output JSON:

```
{
  "catalog": {
    "product": "Laptop",
```

```
        "product": "Smartphone",
        "product": "Tablet"
    }
}
```

7.3 Test 3: Imbricare pe mai multe niveluri (Deep Nesting)

Verifică adâncimea arborelui de parsare și menținerea indentării corecte pe 4+ niveluri.

Input XML:

```
<company>
  <department>
    <name>IT</name>
    <team>
      <lead>Alice</lead>
      <project>
        <title>XML Translator</title>
        <status>Completed</status>
      </project>
    </team>
  </department>
</company>
```

Output JSON:

```
{
  "company": {
    "department": {
      "name": "IT",
      "team": {
        "lead": "Alice",
        "project": {
          "title": "XML Translator",
          "status": "Completed"
        }
      }
    }
  }
}
```

7.4 Test 4: Elemente cu conținut mixt (Mixed Content)

Acest test observă modul în care translatorul tratează datele de tip text în interiorul unor elemente care conțin la rândul lor alte obiecte.

Input XML:

```
<note>
  <to>George</to>
  <from>Antonio</from>
  <heading>Reminder</heading>
  <body>Do not forget the project!</body>
</note>
```


Output JSON:

```
{
  "note": {
    "to": "George",
    "from": "Antonio",
    "heading": "Reminder",
    "body": "Do not forget the project!"
  }
}
```

7.5 Test 5: Structură pentru date de configurare

Un test util pentru a vedea cum translatorul poate fi folosit pentru a converti fișiere de setări din XML în JSON (format preferat în aplicațiile web moderne).

Input XML:

```
<config>
  <server>
    <ip>127.0.0.1</ip>
    <port>8080</port>
  </server>
  <database>
    <user>admin</user>
    <pass>secret123</pass>
  </database>
</config>
```

Output JSON:

```
{
  "config": {
    "server": {
      "ip": "127.0.0.1",
      "port": "8080"
    },
    "database": {
      "user": "admin",
      "pass": "secret123"
    }
  }
}
```

8. Concluzii și Îmbunătățiri Propuse

Proiectul demonstrează eficiența instrumentelor Lex/Yacc în procesarea limbajelor ierarhice.

Îmbunătățiri:

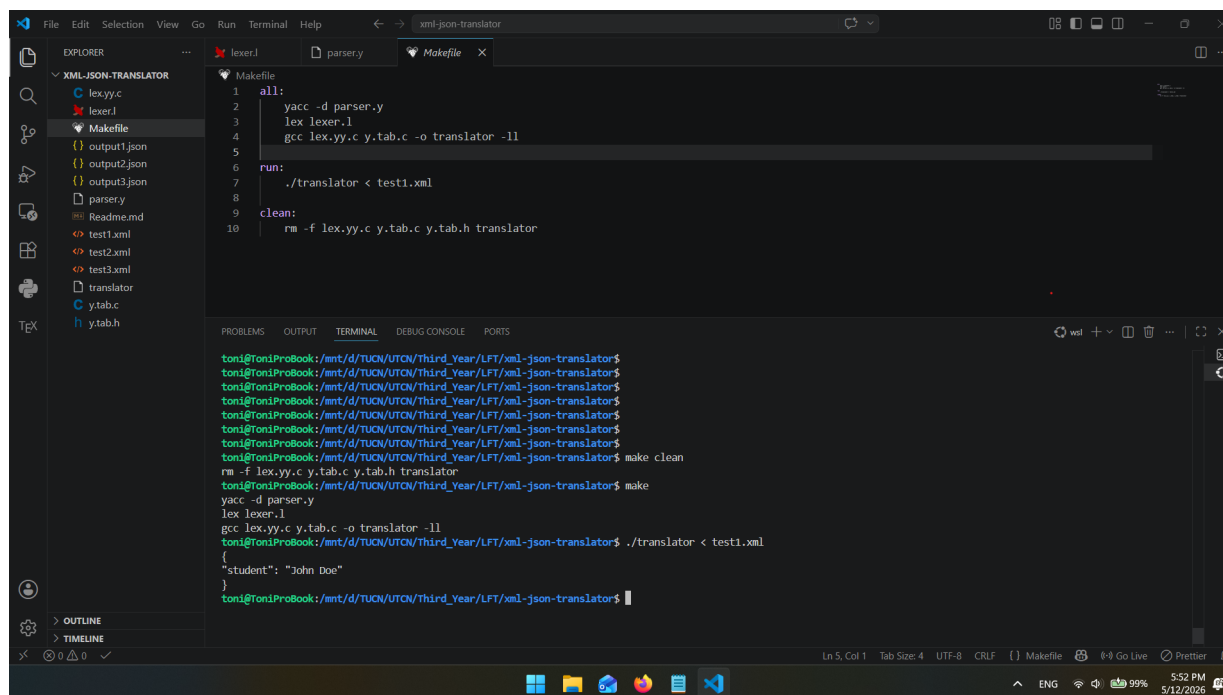
1. Detecția automată a tipurilor (Number vs String).
2. Suport pentru attribute XML sub formă de chei "@attr".
3. Suport pentru namespace-uri XML.

9. Rulări și rezultate (Capturi de ecran)

Această secțiune documentează procesul de compilare și execuție a translatorului, oferind dovezi vizuale ale funcționării corecte a acestuia pe structuri XML complexe.

9.1 Compilarea proiectului

Pentru generarea executabilului, s-a utilizat utilitarul `make`. Acesta a orchestrat rularea `yacc` pentru parser, `lex` pentru analizorul lexical și `gcc` pentru legarea obiectelor într-un fișier binar numit `translator`.



```
Makefile
1 all:
2     yacc -d parser.y
3     lex lexer.l
4     gcc lex.yy.c y.tab.c -o translator -ll
5
6 run:
7     ./translator < test1.xml
8
9 clean:
10    rm -f lex.yy.c y.tab.c y.tab.h translator

toni@ToniProBook: /mnt/d/TUCN/UTCN/Third_Year/LFT/xml-json-translator$
toni@ToniProBook: /mnt/d/TUCN/UTCN/Third_Year/LFT/xml-json-translator$
toni@ToniProBook: /mnt/d/TUCN/UTCN/Third_Year/LFT/xml-json-translator$
toni@ToniProBook: /mnt/d/TUCN/UTCN/Third_Year/LFT/xml-json-translator$
toni@ToniProBook: /mnt/d/TUCN/UTCN/Third_Year/LFT/xml-json-translator$
toni@ToniProBook: /mnt/d/TUCN/UTCN/Third_Year/LFT/xml-json-translator$
toni@ToniProBook: /mnt/d/TUCN/UTCN/Third_Year/LFT/xml-json-translator$
toni@ToniProBook: /mnt/d/TUCN/UTCN/Third_Year/LFT/xml-json-translator$ make clean
rm -f lex.yy.c y.tab.c y.tab.h translator
toni@ToniProBook: /mnt/d/TUCN/UTCN/Third_Year/LFT/xml-json-translator$ make
yacc -d parser.y
lex lexer.l
gcc lex.yy.c y.tab.c -o translator -ll
toni@ToniProBook: /mnt/d/TUCN/UTCN/Third_Year/LFT/xml-json-translator$ ./translator < test1.xml
{
  "student": "John Doe"
}
toni@ToniProBook: /mnt/d/TUCN/UTCN/Third_Year/LFT/xml-json-translator$
```

Figura 1: Procesul de compilare a componentelor Lex și Yacc.

9.2 Execuția pe scenariul complex (Company)

S-a testat translatorul folosind un fișier XML cu mai multe niveluri de imbricare (Company → Department → Team → Project). Comanda de rulare implică redirectionarea fișierului de intrare către intrarea standard a programului.

```
./translator < test_complex.xml > output_complex.json
```

Listing 3: Comanda de rulare in terminal

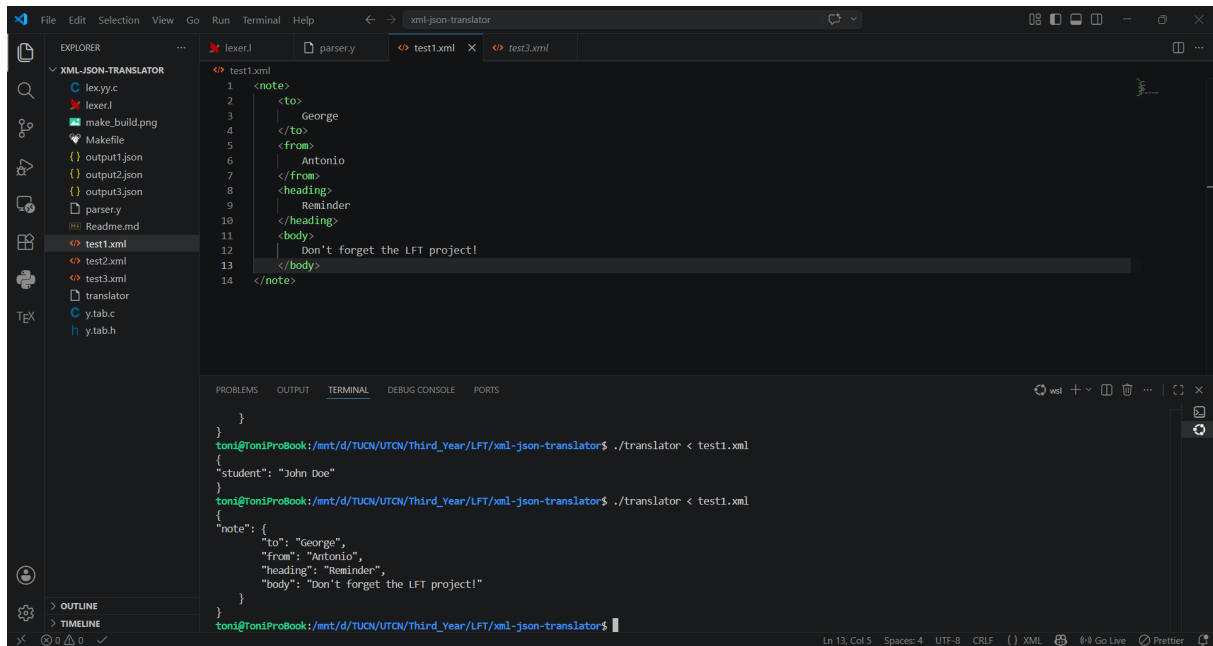
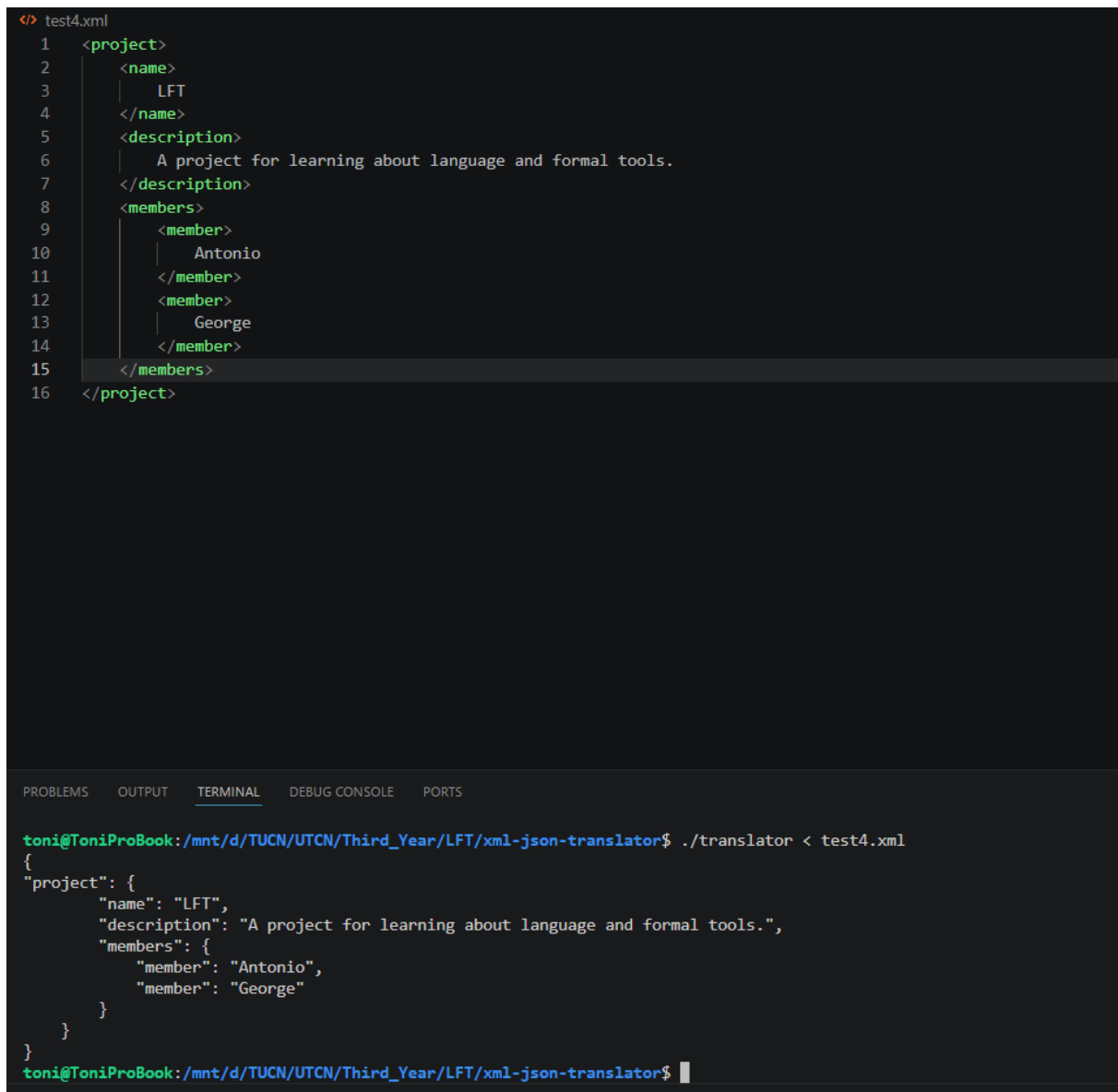


Figura 2: Execuția translatorului pe un fișier XML cu ierarhie adâncă.

```
./translator < test.xml > output.json
```

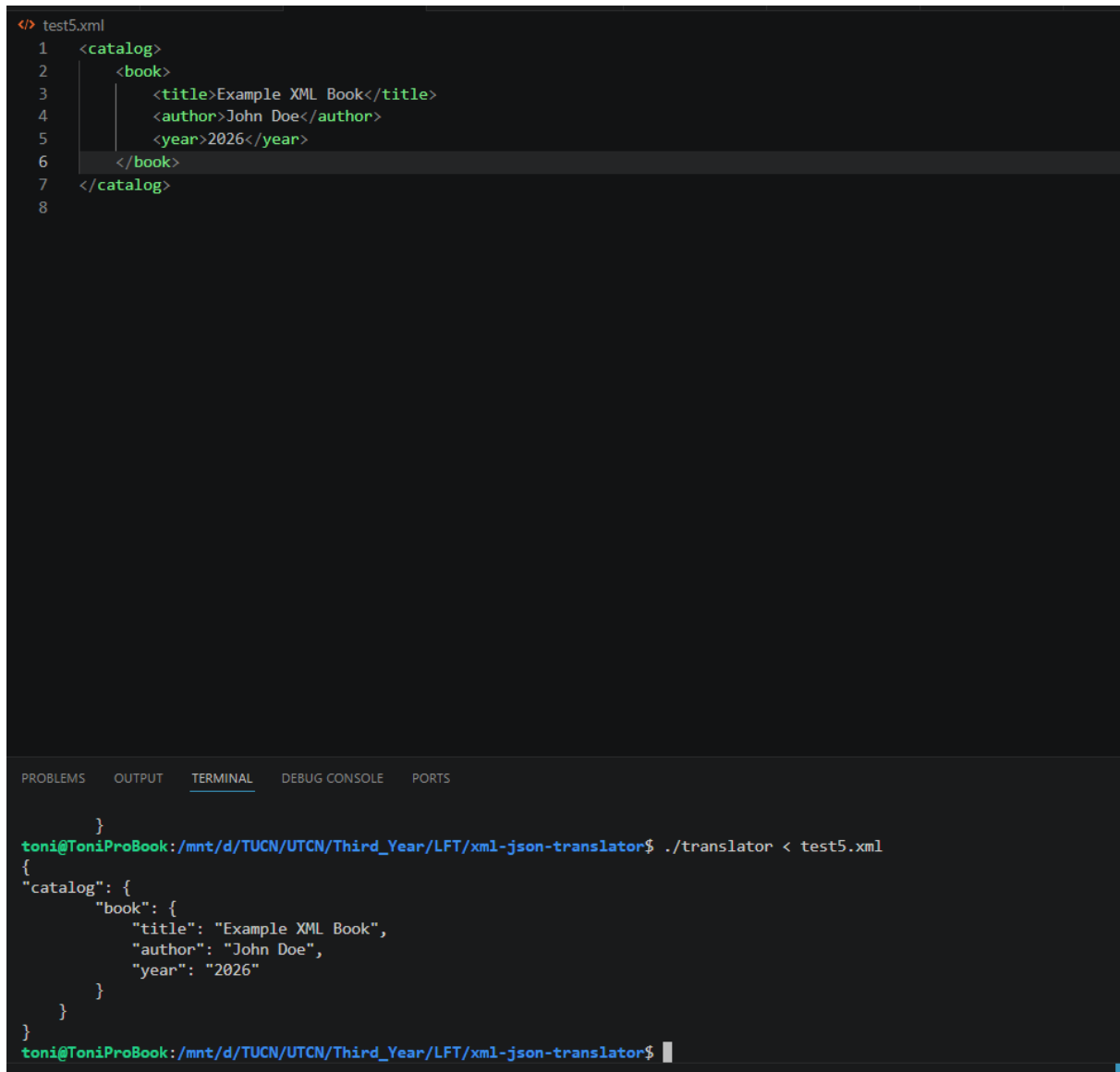
Listing 4: Comanda de rulare in terminal



```
<> test4.xml
1  <project>
2    <name>
3      LFT
4    </name>
5    <description>
6      A project for learning about language and formal tools.
7    </description>
8    <members>
9      <member>
10         Antonio
11      </member>
12      <member>
13         George
14      </member>
15    </members>
16  </project>
```

```
toni@ToniProBook:/mnt/d/TUCN/UTCN/Third_Year/LFT/xml-json-translator$ ./translator < test4.xml
{
  "project": {
    "name": "LFT",
    "description": "A project for learning about language and formal tools.",
    "members": {
      "member": "Antonio",
      "member": "George"
    }
  }
}
toni@ToniProBook:/mnt/d/TUCN/UTCN/Third_Year/LFT/xml-json-translator$
```

Figura 3: Execuția translatorului pe un fișier XML cu ierarhie adâncă.

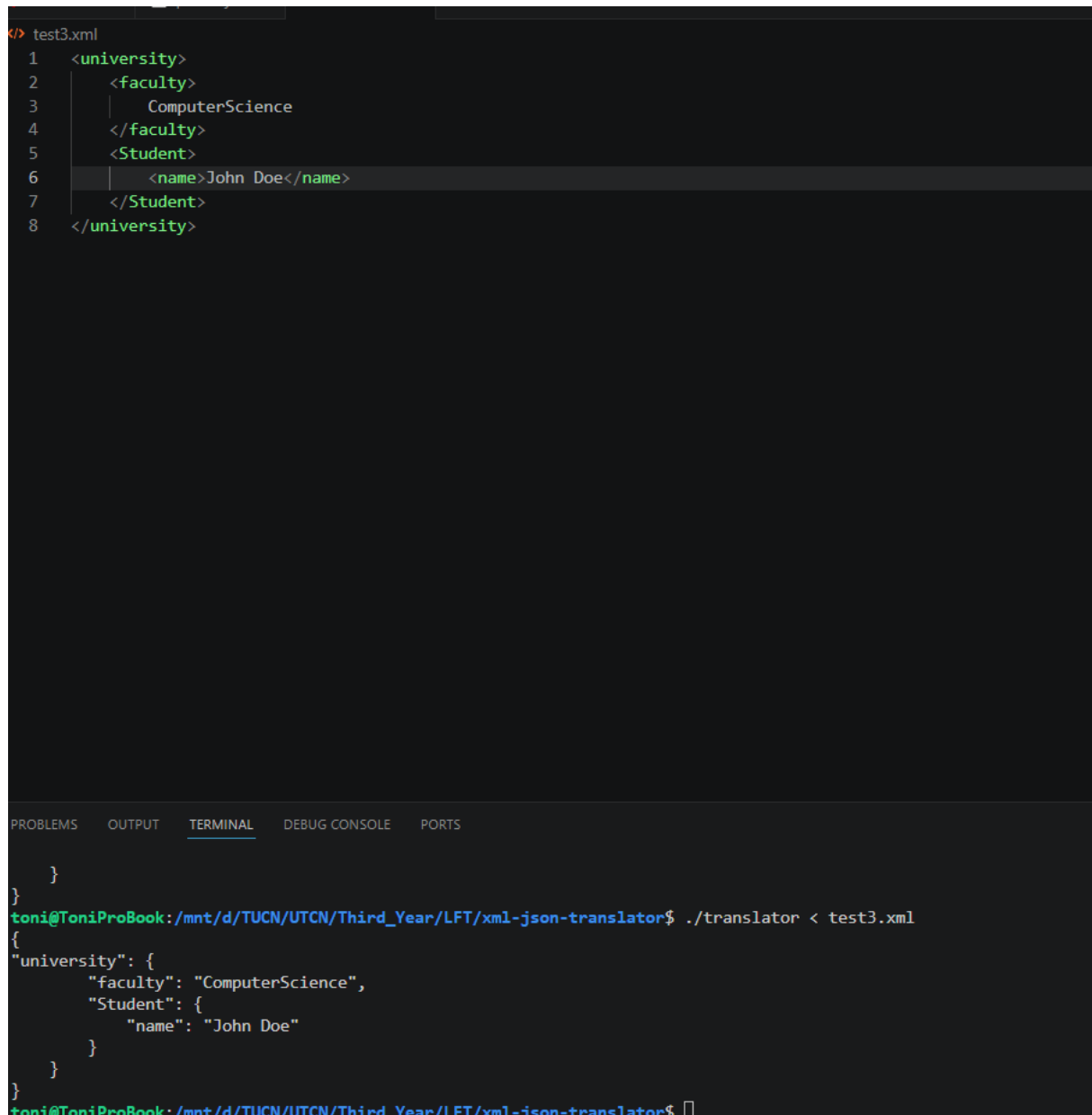


```
</> test5.xml
1  <catalog>
2    <book>
3      <title>Example XML Book</title>
4      <author>John Doe</author>
5      <year>2026</year>
6    </book>
7  </catalog>
8
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS

```
}
toni@ToniProBook: /mnt/d/TUCN/UTCN/Third_Year/LFT/xml-json-translator$ ./translator < test5.xml
{
  "catalog": {
    "book": {
      "title": "Example XML Book",
      "author": "John Doe",
      "year": "2026"
    }
  }
}
toni@ToniProBook: /mnt/d/TUCN/UTCN/Third_Year/LFT/xml-json-translator$
```

Figura 4: Execuția translatorului pe un fișier XML.



```
</> test3.xml
1  <university>
2    <faculty>
3      ComputerScience
4    </faculty>
5    <Student>
6      <name>John Doe</name>
7    </Student>
8  </university>
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS

```
}
}
toni@ToniProBook: /mnt/d/TUCN/UTCN/Third_Year/LFT/xml-json-translator$ ./translator < test3.xml
{
  "university": {
    "faculty": "ComputerScience",
    "Student": {
      "name": "John Doe"
    }
  }
}
toni@ToniProBook: /mnt/d/TUCN/UTCN/Third_Year/LFT/xml-json-translator$
```

Figura 5: Execuția translatorului pe un fișier XML.